Métodos Iterativos para Sistemas Lineales

Método de Jacobi

Diego Valencia

Universidad Mariana

11 de marzo de 2026

Contenido

1. Introducción
2. Formulación general
3. Método de Jacobi
4. Convergencia: ¿cuándo funcionan?
5. Ejemplo guiado (a mano)
6. Comparación y conclusiones
7. Ejercicios
8. Códigos

¿Por qué métodos iterativos?

¿Cuándo conviene resolver $A\mathbf{x} = \mathbf{b}$ de forma iterativa?

Motivación

Contexto

Resolver un sistema lineal $A\mathbf{x} = \mathbf{b}$ es una tarea central en ingeniería. En muchos casos, los métodos directos (Gauss, LU) pueden ser costosos o poco convenientes.

- Sistemas **grandes** (miles o millones de incógnitas).
- Matrices **dispersas** (sparse): muchos ceros.
- Soluciones aproximadas son aceptables (tolerancia).
- Problemas repetitivos (múltiples $\mathbf{b}$ con el mismo A).

Idea clave

Construir una sucesión $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots$ que converja a la solución.

Ejemplos en Ingeniería Civil

- **Estructuras**: métodos matriciales (rigidez) $\Rightarrow$ sistemas grandes y dispersos.
- **Hidráulica/CFD**: discretización (volúmenes finitos, diferencias finitas) $\Rightarrow$ A sparse.
- **Geotecnia**: elementos finitos para deformaciones y consolidación.
- **Redes**: caudales/presiones en redes de tuberías (iteraciones acopladas).

Sistema lineal y notación

Problema

$$A\mathbf{x} = \mathbf{b}, \qquad A \in \mathbb{R}^{n\times n},\ \mathbf{x} \in \mathbb{R}^n,\ \mathbf{b} \in \mathbb{R}^n.$$

Criterios de parada (prácticos)

Tres opciones comunes

- **Error relativo aproximado**:

$$\frac{\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|}{\|\mathbf{x}^{(k+1)}\|} < \text{tol}$$

- **Residual**:

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}, \qquad \frac{\|\mathbf{r}^{(k)}\|}{\|\mathbf{b}\|} < \text{tol}$$

- **Máximo de iteraciones**: $k \leq k_{\text{máx}}$.

Recomendación en clase

Usar el **residual** como indicador principal (mide qué tan bien satisface el sistema).

Idea del método de Jacobi

Despeje por ecuación

Para cada ecuación i:

$$a_{ii}x_i = b_i - \sum_{j \neq i} a_{ij}x_j \quad \Rightarrow \quad x_i = \frac{1}{a_{ii}}\left(b_i - \sum_{j \neq i} a_{ij}x_j\right)$$

Regla Jacobi

En Jacobi, todas las componentes se actualizan usando **solo valores de la iteración anterior**:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j \neq i} a_{ij}\, x_j^{(k)}\right)$$

- Fácil de implementar.
- Naturalmente paralelizable.
- Puede converger más lento que Gauss–Seidel.

Condiciones típicas de convergencia

Suficientes (muy usadas en curso)

- **Dominancia diagonal estricta** por filas:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \forall i$$

- **Matriz simétrica definida positiva (SPD)**:

$$A = A^T, \qquad \mathbf{x}^T A \mathbf{x} > 0 \;\; \forall \mathbf{x} \neq \mathbf{0}$$

Comentarios

- Si A es diagonalmente dominante estricta: **Jacobi y GS convergen**.
- Si A es SPD: **GS converge** (y usualmente muy bien).

¿Y si no converge? (ideas prácticas)

- Reordenar ecuaciones/variables para favorecer dominancia diagonal.
- Escalamiento (normalización) del sistema.

entre otros.

En clase

Primero verificar dominancia diagonal y luego aplicar Jacobi con criterio de parada por residual.

Sistema de ejemplo (y residual)

Problema

Resolver aproximadamente el sistema:

$$\begin{cases} 10x_1 - x_2 + 2x_3 = 6 \\ -x_1 + 11x_2 - x_3 = 25 \\ 2x_1 - x_2 + 10x_3 = -11 \end{cases}$$

- La matriz es diagonalmente dominante estricta:

$$10 > 1 + 2, \quad 11 > 1 + 1, \quad 10 > 2 + 1$$

- Por tanto, **Jacobi debería converger**.
- Tomar $\mathbf{x}^{(0)} = (0, 0, 0)^T$.

Residual (medida del error del sistema)

$$r^{(k)} = b - Ax^{(k)}, \qquad \|r^{(k)}\|_\infty = \max_i |r_i^{(k)}|$$

Jacobi: fórmulas de iteración + residual

Despejes

$$x_1 = \frac{1}{10}\left(6 + x_2 - 2x_3\right), \quad x_2 = \frac{1}{11}\left(25 + x_1 + x_3\right), \quad x_3 = \frac{1}{10}\left(-11 - 2x_1 + x_2\right)$$

Jacobi

$$x_1^{(k+1)} = \frac{1}{10}\left(6 + x_2^{(k)} - 2x_3^{(k)}\right)$$

$$x_2^{(k+1)} = \frac{1}{11}\left(25 + x_1^{(k)} + x_3^{(k)}\right)$$

$$x_3^{(k+1)} = \frac{1}{10}\left(-11 - 2x_1^{(k)} + x_2^{(k)}\right)$$

Cálculo del residual en cada iteración

$$r^{(k)} = \begin{bmatrix} 6 - (10x_1^{(k)} - x_2^{(k)} + 2x_3^{(k)}) \\ 25 - (-x_1^{(k)} + 11x_2^{(k)} - x_3^{(k)}) \\ -11 - (2x_1^{(k)} - x_2^{(k)} + 10x_3^{(k)}) \end{bmatrix}, \qquad \|r^{(k)}\|_\infty = \text{máx}\,|r_i^{(k)}|$$

Jacobi: iteraciones y error residual

Iteración 0 (semilla)

$$x^{(0)} = (0,0,0)^T \Rightarrow r^{(0)} = b - Ax^{(0)} = b = \begin{bmatrix} 6 \\ 25 \\ -11 \end{bmatrix}$$

$$\|r^{(0)}\|_\infty = \text{máx}\{6, 25, 11\} = 25$$

Iteración 1

Con $\mathbf{x}^{(0)} = (0,0,0)$:

$$x_1^{(1)} = 0{,}6, \quad x_2^{(1)} = 2{,}2727, \quad x_3^{(1)} = -1{,}1$$

Residual en $k = 1$

$$r_1^{(1)} = 6 - (10(0{,}6) - 2{,}2727 + 2(-1{,}1)) = 6 - (1{,}5273) = 4{,}4727$$

$$r_2^{(1)} = 25 - (-0{,}6 + 11(2{,}2727) - (-1{,}1)) = 25 - (25{,}5000) = -0{,}5000$$

$$r_3^{(1)} = -11 - (2(0{,}6) - 2{,}2727 + 10(-1{,}1)) = -11 - (-12{,}0727) = 1{,}0727$$

$$\|r^{(1)}\|_\infty = \text{máx}\{4{,}4727, 0{,}5000, 1{,}0727\} = 4{,}4727$$

Jacobi: iteración 2 y error residual

Iteración 2

Usando $\mathbf{x}^{(1)}$:

$$x_1^{(2)} = 1{,}0473, \quad x_2^{(2)} = 2{,}1364, \quad x_3^{(2)} = -0{,}9927$$

Residual en $k = 2$

$$r_1^{(2)} = 6 - (10(1{,}0473) - 2{,}1364 + 2(-0{,}9927)) = 6 - (6{,}3509) = -0{,}3509$$

$$r_2^{(2)} = 25 - (-1{,}0473 + 11(2{,}1364) - (-0{,}9927)) = 25 - (25{,}5399) = -0{,}5399$$

$$r_3^{(2)} = -11 - (2(1{,}0473) - 2{,}1364 + 10(-0{,}9927)) = -11 - (-9{,}9691) = -1{,}0309$$

$$\|r^{(2)}\|_\infty = \text{máx}\{0{,}3509, 0{,}5399, 1{,}0309\} = 1{,}0309$$

El residual disminuye: $25 \to 4{,}4727 \to 1{,}0309$ (mejora de la aproximación).

Ejemplo guiado: sistema y criterio

Sistema

$$\begin{cases} 10x_1 - x_2 + 2x_3 = 6 \\ -x_1 + 11x_2 - x_3 = 25 \\ 2x_1 - x_2 + 10x_3 = -11 \end{cases} \qquad A = \begin{bmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix}, \; b = \begin{bmatrix} 6 \\ 25 \\ -11 \end{bmatrix}$$

Residual (indicador de calidad)

$$r^{(k)} = b - Ax^{(k)}, \qquad \|r^{(k)}\|_\infty = \max_i |r_i^{(k)}|$$

- Semilla: $x^{(0)} = (0, 0, 0)^T$.
- La matriz es diagonalmente dominante $\Rightarrow$ se espera convergencia.

Jacobi: tabla de iteraciones (con residual)

Jacobi (valores calculados y redondeados)

$$x_1^{(k+1)} = \frac{1}{10}(6 + x_2^{(k)} - 2x_3^{(k)}),\ x_2^{(k+1)} = \frac{1}{11}(25 + x_1^{(k)} + x_3^{(k)}),\ x_3^{(k+1)} = \frac{1}{10}(-11 - 2x_1^{(k)} + x_2^{(k)})$$

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$	$\|r^{(k)}\|_\infty$
0	0.00000	0.00000	0.00000	2.50000e+01
1	0.60000	2.27273	-1.10000	4.47273e+00
2	1.04727	2.22727	-0.99273	9.40000e-01
3	1.02127	2.27769	-1.08673	2.38413e-01
4	1.04511	2.26678	-1.07649	5.85917e-02
5	1.04197	2.26988	-1.08235	1.48168e-02

Nota: $r^{(k)} = b - Ax^{(k)}$. Aquí se reporta la norma infinito (máximo error por ecuación).

Comparación rápida del ejemplo

Comparación por residual

- En $k = 1$: Jacobi $\|r^{(1)}\|_\infty \approx 4{,}4$ vs. GS $\approx 4{,}3$.
- En $k = 3$: Jacobi $\approx 0{,}2384$ vs. GS $\approx 0{,}00334$.

Conclusiones

Mensajes clave

- Los métodos iterativos construyen $\mathbf{x}^{(k)}$ hasta cumplir tolerancia.
- Jacobi es simple y paralelizable, pero suele ser más lento.
- La convergencia se facilita si A es **diagonalmente dominante** o **SPD**.
- En práctica: monitorear el **residual** y usar $k_{\text{máx}}$.

Ejercicios propuestos

1. Verifique si la matriz es diagonalmente dominante y aplique **Jacobi** (2 iteraciones):

$$\begin{cases} 8x_1 + x_2 - x_3 = 7 \\ 2x_1 + 10x_2 + x_3 = -8 \\ -x_1 + x_2 + 5x_3 = 6 \end{cases}$$

2. Para el sistema del ejemplo guiado, calcule el residual $\mathbf{r}^{(2)} = \mathbf{b} - A\mathbf{x}^{(2)}$ para Jacobi.

Python: Método de Jacobi

Objetivo: resolver $A\mathbf{x} = \mathbf{b}$ por Jacobi con criterio por residual.

```
import numpy as np

def jacobi(A, b, x0=None, tol=1e-8, kmax=200):
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = len(b)

    x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
    D = np.diag(A)
    R = A - np.diagflat(D)     # R = L + U

    for k in range(kmax):
        x_new = (b - R @ x) / D
        r = b - A @ x_new
        if np.linalg.norm(r, ord=np.inf) / np.linalg.norm(b, ord=np.inf) < tol:
            return x_new, k+1
        x = x_new.copy()

    return x, kmax

A = [[10, -1,  2],
     [-1, 11, -1],
     [ 2, -1, 10]]
b = [6, 25, -11]

x_aprox, it = jacobi(A, b, x0=[0,0,0], tol=1e-10, kmax=500)
print("x =", x_aprox, " iteraciones =", it)
```

¡Gracias!

¿Preguntas?